Queue ADT for Fair Customer Service

Data Structures and Algorithms – Semester Examination

Problem Statement

Customer service centers receive requests continuously throughout the day. A major challenge is managing high volumes while ensuring absolute fairness.

> **"Requests must be processed in the exact order they arrive."**

- Preventing "line jumping" or skipping.
- Maintaining consistent wait times.
- Transparent processing for all users.

Introduction to Queue ADT

What is an ADT?

An Abstract Data Type is a high-level model that defines values and operations without specifying implementation details.

ADT vs Data Structure

ADT is the "What" (logical contract), while Data Structure is the "How" (physical implementation using arrays or lists).

Queue Definition: A linear ADT following the FIFO (First In, First Out) principle.

FIFO Visual Diagram

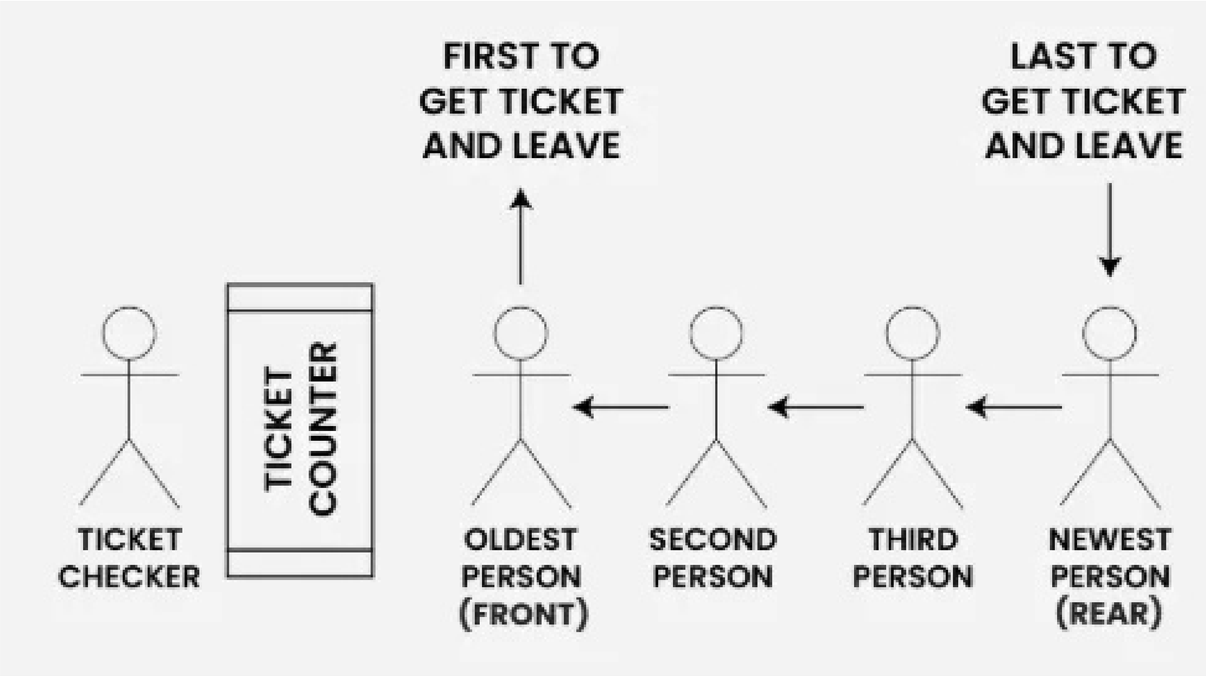
The FIFO Principle

FIFO stands for **First In, First Out**. It is the fundamental logic that drives fairness in service systems.

A → B → C → D

Customer A arrives first, so Customer A is served first. No exceptions are made based on request size or status.

> **First request received = First request processed.**



Core Queue Operations

Enqueue

Adds a new request to the **Rear** of the queue.

Dequeue

Removes the oldest request from the **Front**.

Peek

Views the element at the **Front** without removing it.

IsEmpty

Returns true if there are no pending requests to serve.

How Queue Ensures Fairness

Correct (FIFO)

A → B → C → D

- Strict arrival order.
- Predictable wait times.
- No starvation of older tasks.

Incorrect (Unfair)

A → C → B → D

- "C" jumped ahead of "B".
- Bypassing the system rules.
- Causes customer frustration.

Real-Life Applications

Call Centers

Bank Tokens

Printer Spooler

CPU Scheduling

Ticket Booking

Web Servers

Algorithm Pseudocode

```
FUNCTION Enqueue(x)
  IF queue is FULL THEN
    RETURN "Overflow"
  ELSE
    REAR = REAR + 1
    QUEUE[REAR] = x
  END IF
END FUNCTION
```

```
FUNCTION Dequeue()
  IF queue is EMPTY THEN
    RETURN "Underflow"
  ELSE
    element = QUEUE[FRONT]
    FRONT = FRONT + 1
    RETURN element
  END IF
END FUNCTION
```

Time Complexity

Operation	Time Complexity	Description
Enqueue	$O(1)$	Direct insertion at the rear index.
Dequeue	$O(1)$	Direct removal/pointer shift at the front.
Peek	$O(1)$	Immediate access to the front element.
IsEmpty	$O(1)$	Simple comparison check (Front == -1).

All operations are constant time because they access specific indices without iterating through the collection.

Core Advantages

Fairness

Guarantees that no customer is ignored or starved of resources regardless of their wait time.

Efficiency

O(1) complexity ensures that the system remains fast even with thousands of requests.

Predictable

Simple flow makes it easy to debug, implement, and maintain in large-scale systems.

Conclusion

The Queue ADT is the most suitable model for customer service because it mirrors natural human fairness.

"First Come, First Served = Fair Service"